Post-Coding Development

Meet me at the PiCcaDilly - where intent becomes implementation

Matthias G. Eckermann

2026-03-25

© 2026, Matthias G. Eckermann, Potsdam, Germany

Section 1

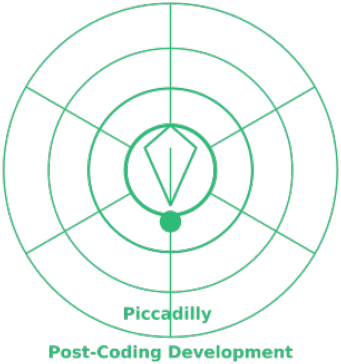
What

© 2026, Matthias G. Eckermann, Potsdam, Germany

Human Intent, Machine Implementation

Domain experts write **specifications**.
AI generates **all** implementation code.
Engineers never write implementation code directly.

Not vibe coding: if the output is wrong,
you fix the **spec** — never the code.



© 2026, Matthias G. Eckermann, Potsdam, Germany

This is not “AI-assisted coding”

	Traditional	Vibe Coding	**PCD**
Human writes	code	code + prompts	**specs only**
AI role	none	suggests	**translates**
Primary artifact	source code	source code	**specification**
Target language	developer	developer	**template**
Regulated domains	manual audit	prohibited	**enabled**

In vibe coding, if the output is wrong, you edit the code.
In PCD, **you never touch the code** — you fix the spec and regenerate.

© 2026, Matthias G. Eckermann, Potsdam, Germany

The specification

```
## BEHAVIOR: transfer
INPUTS:
  from: Account
  amount: Amount
STEPS:
  1. Validate preconditions.
  2. Debit from.balance.
  3. Credit to.balance.
POSTCONDITIONS:
  - SUM(balances) unchanged
EXAMPLES:
  EXAMPLE: success
  GIVEN: balance = 100
  WHEN:  transfer(30)
  THEN:  balance = 70
```

No programming language.
No target platform.
No toolchain knowledge required.
The **template** decides all of that.

© 2026, Matthias G. Eckermann, Potsdam, Germany

Section 2

Why

© 2026, Matthias G. Eckermann, Potsdam, Germany

1. Separation of concerns

The spec says **what** and **where**.
The template decides **how**:

- programming language
- packaging format
- toolchain conventions
- deployment target

Change language across the whole project?
Change one preset file.
The spec is untouched.
A spec written today is valid in 2045 —
regardless of what language is in fashion.

© 2026, Matthias G. Eckermann, Potsdam, Germany

2. Long-term maintainability

Specs are more stable than code.
Code accumulates technical debt.
Specs describe intent — intent rarely changes.

When requirements change:

- update the spec
- regenerate the code
- no manual refactoring

The spec is the documentation.
Not a comment that drifts from the code.
Not a wiki page nobody updates.
The running system is always derived from the spec.

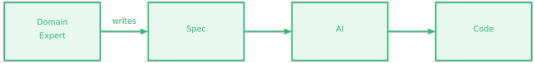
© 2026, Matthias G. Eckermann, Potsdam, Germany

3. Domain experts author directly

Today

Every handoff loses information.
Requirements get simplified.
Misunderstandings go undetected.

With PCD



The cardiologist specifies
the device behaviour directly.
No interpreter in the middle.

© 2026, Matthias G. Eckermann, Potsdam, Germany

And: formal verification is available

The LLM translation is probabilistic — we do not claim otherwise.

For components that **require** stronger guarantees:

Lean 4, F*, or Dafny can be added as an **optional verification layer** — without changing the specification format.
The spec stays the same. The verification path is a template choice.

- memory safety by construction
- formal proofs of invariants
- state machine correctness
- ISO 26262 / DO-178C evidence

© 2026, Matthias G. Eckermann, Potsdam, Germany

Proof: pcd-lint

`pcd-lint` — the specification validator — was **specified and generated using PCD itself.**

Zero hand-written implementation code.

Tested across multiple AI models, three continents, one result:

Every model resolved Go from the template
without being told.

© 2026, Matthias G. Eckermann, Potsdam, Germany

© 2026, Matthias G. Eckermann, Potsdam, Germany

Section 3

How

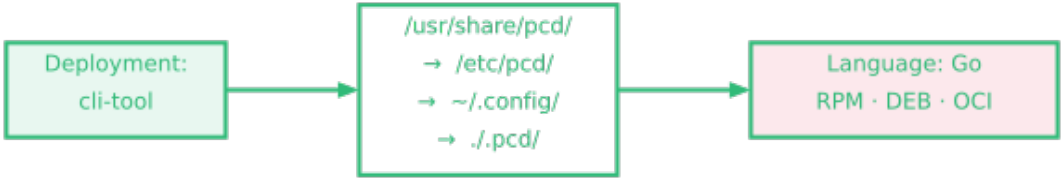
© 2026, Matthias G. Eckermann, Potsdam, Germany

The workflow

1. Domain expert writes a spec (or AI interviews them)
2. `pcd-lint` validates structure
3. Deployment template resolves the language
4. LLM translates spec → code + audit bundle

© 2026, Matthias G. Eckermann, Potsdam, Germany

Language is never your problem



The spec declares **what** and **where** (deployment context).

The template resolves **language, packaging, conventions**.

A spec written today is valid if the organisation
switches from Go to Rust in 2045 — **no spec change needed**.

© 2026, Matthias G. Eckermann, Potsdam, Germany

The audit bundle

Every translation produces:

- specification (human-reviewable)
- generated source code
- packaging artifacts (RPM, DEB, OCI)
- independent test suite
- translation report
- workflow diagram (Pikchr)
- metadata + hashes

Designed for:

- ISO 26262 (automotive)
- DO-178C (aviation)
- IEC 62304 (medical)
- Common Criteria EAL4+
- EU Cyber Resilience Act

© 2026, Matthias G. Eckermann, Potsdam, Germany

Getting started

Write a spec (no format knowledge needed):

```
# AI interviews the domain expert:
ollama run llama3.2 \
  "$(cat prompts/interview-prompt.md)"
```

Then validate:

```
pcd-lint myspec.md
```

Translate to code:

```
# mcphost with the translator prompt
# reads template + spec,
# produces code + audit bundle
```

Everything in the repo:
github.com/mge1512/pcd

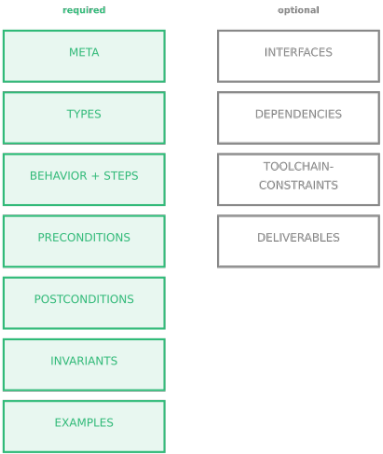
© 2026, Matthias G. Eckermann, Potsdam, Germany

Section 4

The specification

© 2026, Matthias G. Eckermann, Potsdam, Germany

Spec sections



`pcd-lint` validates structure before any translation begins.

Key constraints:

- BEHAVIOR must include `STEPS:`
- INVARIANTS carry `[observable]` or `[implementation]` tags
- EXAMPLES must cover error paths
- No programming language in spec

© 2026, Matthias G. Eckermann, Potsdam, Germany

BEHAVIOR in depth

```
## BEHAVIOR: shutdown
Constraint: required
INPUTS:
  vm: VirtualMachine
PRECONDITIONS:
  - vm.state = Running
STEPS:
  1. Send shutdown signal.
  2. Record shutdown-start time.
     MECHANISM: do NOT block;
     requeue immediately.
  3. On re-entry: check elapsed.
     If > timeout: Destroy(vm).
POSTCONDITIONS:
  - vm.state = Stopped
```

`STEPS` = the algorithm, not just the contract.

`MECHANISM:` = where the *how* matters for correctness.

`Constraint:` field:

- `required` — always implement
- `supported` — preset activates it
- `forbidden` — never implement

© 2026, Matthias G. Eckermann, Potsdam, Germany

INTERFACES: testable abstractions

```
## INTERFACES

Transport {
  required-methods:
    Connect(ctx, spec) -> (Session, error)
  implementations-required:
    production:  RealTransport
    test-double: FakeTransport {
      configurable: ConnectErr, Sessions
    }
}
```

Independent tests use **only** declared test doubles.
`go test ./independent_tests/` runs without any live service.

© 2026, Matthias G. Eckermann, Potsdam, Germany

© 2026, Matthias G. Eckermann, Potsdam, Germany

Section 5

Templates and translation

© 2026, Matthias G. Eckermann, Potsdam, Germany

Deployment templates

Templates encode **everything** the spec does not:

- target language + alternatives
- binary type, packaging formats
- CLI conventions
- security constraints
- TYPE-BINDINGS
- delivery phases (EXECUTION)
- compile gate

One preset file changes language for the whole project. The spec is untouched.

© 2026, Matthias G. Eckermann, Potsdam, Germany

TYPE-BINDINGS: no translator discretion

Template excerpt:

```
## TYPE-BINDINGS
| Spec type   | LANGUAGE=Go            |
|-------------|------------------------|
| Duration    | metav1.Duration        |
| Timestamp   | metav1.Time            |
| Condition   | metav1.Condition       |
| List<T>     | []T                    |
```

Without TYPE-BINDINGS: one translator produces `string`, another produces `metav1.Duration` — incompatible at the API level.

With TYPE-BINDINGS: **deterministic, language-specific types** — applied mechanically, not by translator judgement.

© 2026, Matthias G. Eckermann, Potsdam, Germany

The translation phases

1 Core implementation

2 Build + packaging

3 Test infrastructure

4 Documentation

5 Compile gate

Defined in the template's `## EXECUTION` section — **not** in the prompt.

Resume logic: partial runs restart from the first missing file.

Compile gate is language-specific:

```
go mod tidy    # direct deps only
               # no fabricated versions
go build ./...
```

`TRANSLATION_REPORT.md` is **always last** — its presence signals a complete run.

© 2026, Matthias G. Eckermann, Potsdam, Germany

© 2026, Matthias G. Eckermann, Potsdam, Germany

Section 6

Tooling

© 2026, Matthias G. Eckermann, Potsdam, Germany

pcd-lint

What it validates:

- required sections present
- META fields + SPDX license
- deployment template resolves
- BEHAVIOR has STEPS
- INVARIANTS have tags
- EXAMPLES cover error paths
- Constraint: field values
- cross-section consistency

Self-hosting:

`pcd-lint` was specified and generated using PCD itself.

Running `pcd-lint` against its own spec passes — the paradigm validates itself.

```
pcd-lint myspec.md
pcd-lint strict=true myspec.md
pcd-lint list-templates
```

© 2026, Matthias G. Eckermann, Potsdam, Germany

Translation confidence

Every `TRANSLATION_REPORT.md` includes:

EXAMPLE	Confidence	Verification method	Unverified claims
vm-start	High	`TestVMStart` — FakeSession	libvirt UUID format
graceful-stop	Medium	`TestGracefulStop` pass 1 only	timeout path
host-ready	Low	none	entire sequence

- **High** = named test in `independent_tests/` passes without live services
- **Medium** = partial test coverage
- **Low** = reasoning only — no test

Unverified claims must be listed **explicitly**. Never silently omitted.

© 2026, Matthias G. Eckermann, Potsdam, Germany

Large projects

Architect defines interfaces first:

```
## INTERFACE
EXPORTS:
  TYPES:  Account, TransferResult
  BEHAVIOR: transfer
  INVARIANTS:
    - GLOBAL: balance >= 0
```

Component specs declare imports:

```
Imports:
  - account-svc: ./account.md
                 #INTERFACE@>=1.2.0
```

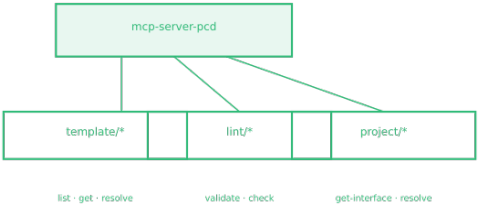
Rules:

- interfaces before implementation
- build order = topological sort
- no circular dependencies
- component fits in one LLM context window

pcd-lint v2 (planned): validates full project graph — imports, circular deps, interface versioning.

© 2026, Matthias G. Eckermann, Potsdam, Germany

mcp-server-pcd



Any MCP-capable host connects directly:

- **mcphost** (CLI)
- **VS Code** with Copilot
- **Claude Desktop**
- custom agents

Preset layering server-side:
vendor → `/etc/pcd-server/` → team → user

The LLM is the wizard.
The server is the data layer.

© 2026, Matthias G. Eckermann, Potsdam, Germany